



Advanced Linear Algebra: Foundations to Frontiers

Robert van de Geijn, Margaret Myers

2.3.3 An "algorithm" for computing the SVD

We really should have created a video for this section. Those who have taken our "Programming for Correctness" course will recognize what we are trying to describe here. Regardless, you can safely skip this unit without permanent (or even temporary) damage to your linear algebra understanding.

In this unit, we show how the insights from the last unit can be molded into an "algorithm" for computing the SVD. We put algorithm in quotes because while the details of the algorithm mathematically exist, they are actually very difficult to compute in practice. So, this is *not* a practical algorithm. We will not discuss a practical algorithm until the very end of the course, in [Week 11](#).

We observed that, starting with matrix A , we can compute one step towards the SVD. If we overwrite A with the intermediate results, this means that after one step

$$\begin{pmatrix} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} \tilde{u}_1 & \tilde{U}_2 \end{pmatrix}^H \begin{pmatrix} \hat{\alpha}_{11} & \hat{a}_{12}^T \\ \hat{a}_{21} & \hat{A}_{22} \end{pmatrix} \begin{pmatrix} \tilde{v}_1 & \tilde{V}_2 \end{pmatrix} = \begin{pmatrix} \sigma_{11} & 0 \\ 0 & B \end{pmatrix},$$

where $\hat{A}$ allows us to refer to the original contents of A .

In our proof of [Theorem 2.3.1.1](#), we then said that the SVD of B , $B = U_B \Sigma_B V_B^H$ could be computed, and the desired U and V can then be created by computing $U = \tilde{U} U_B$ and $V = \tilde{V} V_B$.

Alternatively, one can accumulate U and V every time a new singular value is exposed. In this approach, you start by setting $U = I_{m \times m}$ and $V = I_{n \times n}$. Upon completing the first step (which computes the first singular value), one multiplies U and V from the right with the computed $\tilde{U}$ and $\tilde{V}$:

$$\begin{aligned} U &:= U \tilde{U} \\ V &:= V \tilde{V}. \end{aligned}$$

Now, every time another singular value is computed in future steps, the corresponding unitary matrices are similarly accumulated into U and V .

To explain this more completely, assume that the process has proceeded for k steps to the point where

$$\begin{aligned}
U &= \left(\begin{array}{c|c} U_L & U_R \end{array} \right) \in \mathbb{C}^{m \times m} && \text{with } U_L \in \mathbb{C}^{m \times k} \\
V &= \left(\begin{array}{c|c} V_L & V_R \end{array} \right) \in \mathbb{C}^{n \times n} && \text{with } V_L \in \mathbb{C}^{n \times k} \\
A &= \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) && \text{with } A_{TL} \in \mathbb{C}^{k \times k},
\end{aligned}$$

where the current contents of A are

$$\begin{aligned}
\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) &= \left(\begin{array}{c|c} U_L & U_R \end{array} \right)^H \left(\begin{array}{c|c} \hat{A}_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right) \left(\begin{array}{c|c} V_L & V_R \end{array} \right) \\
&= \left(\begin{array}{c|c} \Sigma_{TL} & 0 \\ \hline 0 & B \end{array} \right).
\end{aligned}$$

This means that in the current step we need to update the contents of A_{BR} with

$$\tilde{U}^H A \tilde{V} = \left(\begin{array}{c|c} \sigma_{11} & 0 \\ \hline 0 & \tilde{B} \end{array} \right)$$

and update

$$\begin{aligned}
\left(\begin{array}{c|c} U_L & U_R \end{array} \right) &:= \left(\begin{array}{c|c} U_L & U_R \end{array} \right) \left(\begin{array}{c|c} I_{k \times k} & 0 \\ \hline 0 & \tilde{U} \end{array} \right) \\
\left(\begin{array}{c|c} V_L & V_R \end{array} \right) &:= \left(\begin{array}{c|c} V_L & V_R \end{array} \right) \left(\begin{array}{c|c} I_{k \times k} & 0 \\ \hline 0 & \tilde{V} \end{array} \right),
\end{aligned}$$

which simplify to

$$U_{BR} := U_{BR} \tilde{U} \text{ and } V_{BR} := V_{BR} \tilde{V}.$$

At that point, A_{TL} is expanded by one row and column, and the left-most columns of U_R and V_R are moved to U_L and V_L , respectively. If A_{BR} ever contains a zero matrix, the process completes with A overwritten with $\Sigma = U^H \hat{V}$. These observations, with all details, are captured in [Figure 2.3.3.1](#). In that figure, the boxes in yellow are assertions that capture the current contents of the variables. Those familiar with proving loops correct will recognize the first and last such box as the precondition and postcondition for the operation and

$$\begin{aligned}
\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) &= \left(\begin{array}{c|c} U_L & U_R \end{array} \right)^H \left(\begin{array}{c|c} \hat{A}_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right) \left(\begin{array}{c|c} V_L & V_R \end{array} \right) \\
&= \left(\begin{array}{c|c} \Sigma_{TL} & 0 \\ \hline 0 & B \end{array} \right)
\end{aligned}$$

as the loop-invariant that can be used to prove the correctness of the loop via a proof by induction.

$A = \hat{A}$
$U := I_{m \times m}; \quad V := I_{n \times n}$ $A \rightarrow \left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right), U \rightarrow (U_L U_R), V \rightarrow (V_L V_R) \text{ where } A_{TL} \text{ is } 0 \times 0, U_L \text{ is } m \times 0, V_L \text{ is } n \times 0$
$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = (U_L U_R)^H \left(\begin{array}{c c} \hat{A}_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right) (V_L V_R) = \left(\begin{array}{c c} \Sigma_{TL} & 0 \\ \hline 0 & B \end{array} \right) \text{ (here } \Sigma_{TL} \text{ is } 0 \times 0)$
while $\ B\ _2 \neq 0$
$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = (U_L U_R)^H \left(\begin{array}{c c} \hat{A}_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right) (V_L V_R) = \left(\begin{array}{c c} \Sigma_{TL} & 0 \\ \hline 0 & B \end{array} \right) \wedge \ B\ _2 \neq 0$
$\sigma_{11} = \ A_{BR}\ _2$ if $\sigma_{11} = 0$ break (exit loop) pick $\tilde{v}_1$ s.t. $\ \tilde{v}_1\ _2 = 1$ and $\ A_{BR}\tilde{v}_1\ = \ A_{BR}\ _2 (= \sigma_{11})$ $\tilde{u}_1 := A_{BR}\tilde{v}_1 / \sigma_{11}$ pick $\tilde{V}_2$ and $\tilde{U}_2$ s.t. $\tilde{V} = (\tilde{v}_1 \tilde{V}_2)$ and $\tilde{U} = (\tilde{u}_1 \tilde{U}_2)$ are unitary $V_R := V_R \tilde{V}; \quad U_R := U_R \tilde{U}$ $\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \rightarrow \left(\begin{array}{c c} A_{00} & a_{01} \ A_{02} \\ \hline a_{10}^T & \alpha_{11} \ a_{12}^T \\ \hline A_{20} & a_{21} \ A_{22} \end{array} \right), (U_L U_R) \rightarrow (U_0 u_1 \ U_2), (V_L V_R) \rightarrow (V_0 v_1 \ V_2)$ $\left(\begin{array}{c c} \alpha_{11} & a_{12}^T \\ \hline a_{21} & A_{22} \end{array} \right) := \left(\begin{array}{c c} \sigma_{11} & 0 \\ \hline 0 & \tilde{U}_2^H A_{BR} \tilde{V}_2 \end{array} \right)$ $\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \leftarrow \left(\begin{array}{c c} A_{00} & a_{01} \ A_{02} \\ \hline a_{10}^T & \alpha_{11} \ a_{12}^T \\ \hline A_{20} & a_{21} \ A_{22} \end{array} \right), (U_L U_R) \leftarrow (U_0 \ u_1 U_2), (V_L V_R) \leftarrow (V_0 \ v_1 V_2)$
$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = (U_L U_R)^H \left(\begin{array}{c c} \hat{A}_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right) (V_L V_R) = \left(\begin{array}{c c} \Sigma_{TL} & 0 \\ \hline 0 & B \end{array} \right)$
endwhile
$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = (U_L U_R)^H \left(\begin{array}{c c} \hat{A}_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right) (V_L V_R) = \left(\begin{array}{c c} \Sigma_{TL} & 0 \\ \hline 0 & B \end{array} \right) \wedge \ B\ _2 = 0$
$\underbrace{\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right)}_A = \underbrace{(U_L U_R)^H}_{U^H} \underbrace{\left(\begin{array}{c c} \hat{A}_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right)}_{\hat{A}} \underbrace{(V_L V_R)}_V = \underbrace{\left(\begin{array}{c c} \Sigma_{TL} & 0 \\ \hline 0 & 0 \end{array} \right)}_{\Sigma}$

Figure 2.3.3.1. Algorithm for computing the SVD of A , overwriting A with Σ . In the yellow boxes are assertions regarding the contents of the various matrices.

The reason this algorithm is not practical is that many of the steps are easy to state mathematically, but difficult (computationally expensive) to compute in practice. In particular:

- Computing $\|A_{BR}\|_2$ is tricky and as a result, so is computing $\tilde{v}_1$.
- Given a vector, determining a unitary matrix with that vector as its first column is computationally expensive.

- Assuming for simplicity that $m = n$, even if all other computations were free, computing the product $A_{22} := \tilde{U}_2^H A_{BR} \tilde{V}_2$ requires $O((m - k)^3)$ operations. This means that the entire algorithm requires $O(m^4)$ computations, which is prohibitively expensive when n gets large. (We will see that most practical algorithms discussed in this course cost $O(m^3)$ operations or less.)

Later in this course, we will discuss an algorithm that has an effective cost of $O(m^3)$ (when $m = n$).

Ponder This 2.3.3.1. An implementation of the "algorithm" in [Figure 2.3.3.1](#), using our FLAME API for Matlab (FLAME@lab) [\[5\]](#) that allows the code to closely resemble the algorithm as we present it, is given in mySVD.m (Assignments/Week02/matlab/mySVD.m). This implementation depends on routines in subdirectory Assignments/flameatlab being in the path. Examine this code. What do you notice? Execute it with

```
m = 5;
n = 4;
A = rand( m, n );      % create m x n random matrix
[ U, Sigma, V ] = mySVD( A )
```

Then check whether the resulting matrices form the SVD:

```
norm( A - U * Sigma * V' )
```

and whether U and V are unitary

```
norm( eye( n,n ) - V' * V )
norm( eye( m,m ) - U' * U )
```